Transferencia con un presupuesto de CPU

CC3092 | Ian Cumes 23236 · Javier Valladares 23045 | 30/09/2026

1. Datos y protocolo

CIFAR-10: 60,000 imágenes RGB de 32×32, 10 clases balanceadas; 50,000 de train oficial y 10,000 de test. Split estratificado 45,000/5,000 (4,500/500 por clase), test intacto (1,000 por clase); reducción a 4,500 (450 por clase). Semilla 23236. Se elige el checkpoint y la configuración por F1 macro de validación antes de evaluar test una vez por modelo y fracción.

Media RGB del train oficial: 0.4914, 0.4822, 0.4465; desviación: 0.2470, 0.2435, 0.2616. ImageNet: media 0.485/0.456/0.406; desviación 0.229/0.224/0.225. La CNN normaliza con estadísticas del split de 45k. CNN: crop/flip, BN y dropout; VGG: flips fijados por semilla y almacenados. Validación y test tienen transformaciones deterministas.

2. Investigación de PyTorch [2-9]

vgg16(weights=VGG16_Weights.IMAGENET1K_V1) reutiliza pesos ImageNet; weights.transforms() define resize 256, crop 224 y normalización. features contiene cinco bloques; avgpool=AdaptiveAvgPool2d(7,7) adapta la salida; classifier tiene tres Linear. Se conserva el clasificador y se reemplaza solo la última capa por Linear(4096,10).

requires_grad=False congela parámetros; True descongela B5 (features[24:]) o B4+B5 (features[17:]). train()/eval() controlan dropout/BN; eval no desactiva gradientes. no_grad evita el grafo en evaluación/cache. param_groups asigna LR menor al backbone; SGD usa momentum 0.9 y la CNN AdamW. THOP profile(inputs, custom_ops) cuenta MACs; hooks Conv/Linear permiten desglosarlos. Un MAC equivale aquí a dos FLOPs; totales incluye congelados y entrenables solo requires_grad=True.

CUDA: synchronize antes/después del cronómetro; reset_peak_memory_stats y max_memory_allocated para bytes pico de tensores. Hardware real: Intel i9-13900H, 15.64 GiB RAM; Intel Iris Xe sin CUDA. CNN/FE/FT1: 8 hilos; FT2/FT3 e inferencia: 12. Latencia/RSS son CPU; GPU N/D. VGG usa 32×32, manteniendo los píxeles de CIFAR-10; 224×224 sería 49 veces. Dos o tres épocas por configuración limitan convergencia.

3. Nueve iteraciones: mejor epoch por F1 de validación

ID	Bloques/width	LR clf	LR base	Drop	Ep	Acc %	F1 %
CNN1	w=16	0.003	-	0.15	3	62.82	63.19
CNN2	w=24	0.003	-	0.25	3	61.54	60.56
CNN3	w=24	0.001	-	0.35	3	57.06	56.40
FE1	ninguno	0.01	-	0.5	2	66.54	66.07
FE2	ninguno	0.02	-	0.3	2	68.00	67.94
FE3	ninguno	0.005	-	0.5	2	66.14	65.73
FT1	5	0.01	0.0001	0.5	2	68.04	67.70
FT2	4+5	0.01	0.0001	0.5	2	67.82	67.54
FT3	5	0.02	0.0005	0.3	2	69.62	69.47

CNN: 3 epochs; VGG: 2 epochs; batch CNN 256, VGG 512 (FT2/FT3: 1024). Weight decay 1e-4, excepto CNN3 y FE3: 5e-4. Precision, recall, parámetros, tiempos, RAM y métricas por epoch se conservan en el notebook y los CSV.

Ganadores: mejor epoch, memoria y cómputo estimado

Modelo	Mejor epoch	Pico RSS CPU MB	Pico GPU	Entr. nominal TFLOPs
CNN1	3	800	N/D	8.00
FE2	2	2688	N/D	120.95
FT3	2	2724	N/D	131.14

Rendimiento y costo computacional

CC3092 | Ian Cumes 23236 · Javier Valladares 23045 | 30/09/2026

4. Configuraciones finales y recursos

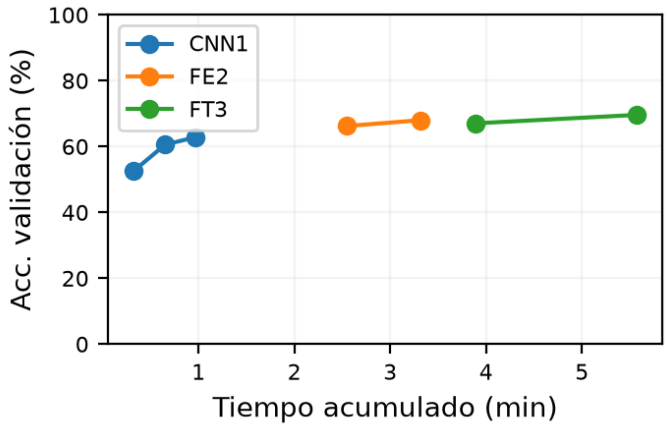
Modelo	Param M	Entr. M	GMAC/img	CPU ms	s/epoch	Total s
CNN1	0.073	0.073	0.010	0.51	18.7	58.1
FE2	134.302	119.587	0.433	16.75	44.2	91.8
FT3	134.302	126.666	0.433	17.99	102.1	208.9

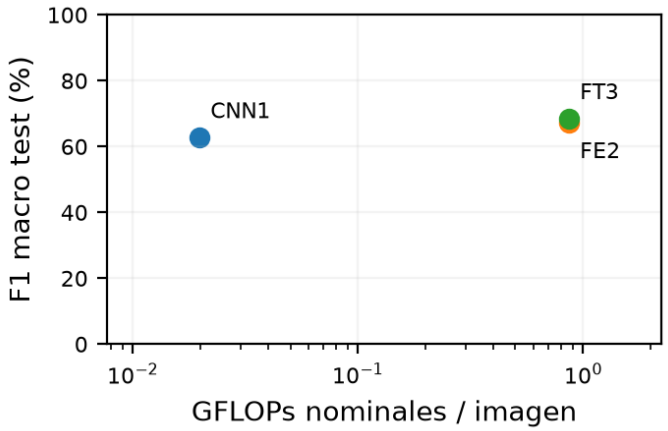
Inferencia original: batch 1, cinco warm-ups y 30 repeticiones. FLOPs=2×MACs. Estimación nominal: N×E×(F_forward+2F_capas entrenables); backward solo en capas entrenables, sin optimizador ni validación. Cache y factorización exacta reducen el cómputo ejecutado respecto a esta estimación, conservando los pesos originales.

Cache compartida: train 97.2s (una vista), validación 10.9s, test 21.8s; cache B4 adicional 22.8s (FT3). Total de entrenamiento en tabla incluye validación/checkpoints; s/epoch es solo entrenamiento. RSS es máximo muestreado del proceso, no una medida de asignaciones CUDA.

5. Test oficial: métricas macro y efecto de reducir datos

Modelo	Datos	Acc %	Precision %	Recall %	F1 %
CNN1	100%	62.39	66.06	62.39	62.84
CNN1_10pct	10%	38.65	41.43	38.65	37.93
FE2	100%	67.32	67.38	67.32	67.27
FE2_10pct	10%	58.97	59.16	58.97	58.89
FT3	100%	68.69	68.86	68.69	68.60
FT3_10pct	10%	57.86	59.69	57.86	57.96



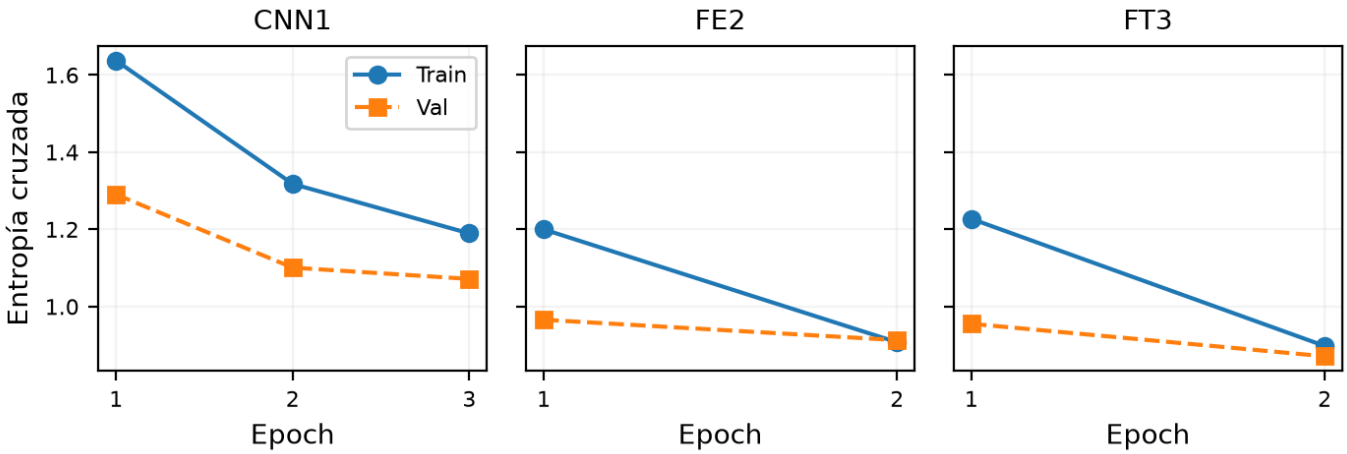


Izquierda: incluye la cache de train/validación en VGG. Derecha: costo de inferencia nominal, no factorizado; las dos estrategias VGG tienen el mismo forward.

Curvas de aprendizaje y errores

CC3092 | Ian Cumes 23236 · Javier Valladares 23045 | 30/09/2026

6. Loss de las configuraciones elegidas



El notebook incluye las nueve curvas train/validación (tres por familia) y las pérdidas por epoch. La búsqueda breve permite observar aprendizaje inicial, pero no establecer el techo de calidad. Dropout y augmentation suelen elevar la loss de entrenamiento; una loss menor en validación no implica por sí sola fuga de datos.

7. Confusiones más frecuentes en test

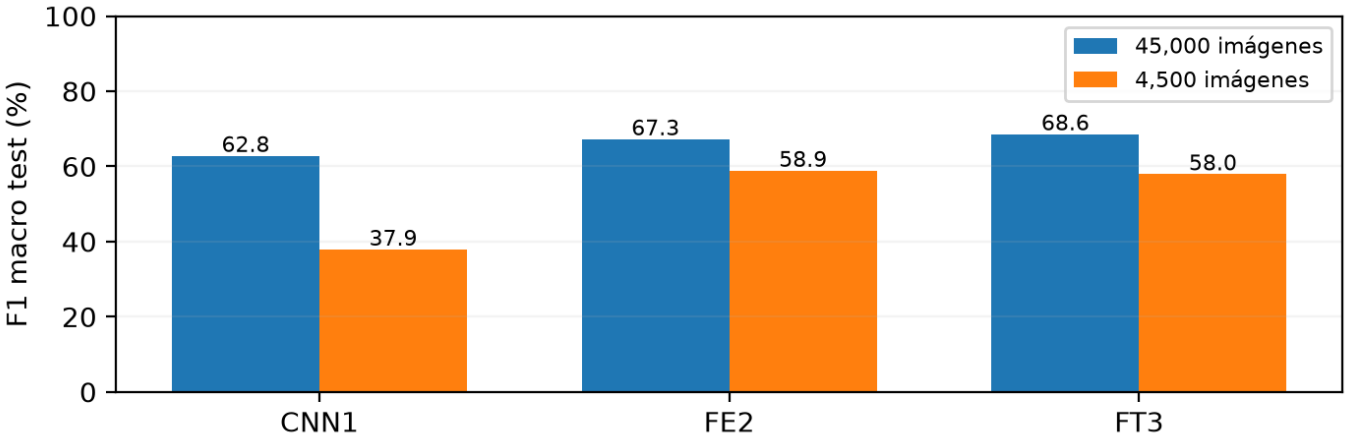
Se cuentan errores en ambas direcciones por par; hay 1,000 ejemplos reales por clase. Las matrices completas, con valores y etiquetas, están en el notebook.

CNN1: cat/dog: 534; airplane/ship: 245; cat/frog: 226

FE2: cat/dog: 323; automobile/truck: 264; bird/frog: 189

FT3: cat/dog: 325; automobile/truck: 246; bird/deer: 199

8. Menos datos, misma configuración y número de épocas



Cambios de F1 (100% menos 10%): CNN propia +24.91 puntos; VGG-16 extractor +8.39 puntos; VGG-16 fine-tuning +10.64 puntos. CNN propia presenta la mayor caída en esta ejecución. Una representación preentrenada suele ser útil con pocos datos y congelar limita grados de libertad; FT requiere LR bajo y control de validación. Aquí conservar épocas también reduce pasos, así que estas diferencias no pueden atribuirse exclusivamente al tamaño de la muestra.

Interpretación y conclusiones

CC3092 | Ian Cumes 23236 · Javier Valladares 23045 | 30/09/2026

Desempeño y costo

VGG-16 fine-tuning obtuvo el mayor F1 macro de test (68.60%) y accuracy 68.69%. La ventaja sobre el segundo fue 1.33 puntos de F1. VGG-16 FT tiene 134.30 M parámetros frente a 0.073 M de la CNN y 43.8 veces sus FLOPs de inferencia. El costo adicional se justifica si se prioriza ese desempeño; una única semilla y dos o tres épocas limitan la fuerza de la comparación.

Relación entre desempeño y recursos

La CNN tiene menor costo de inferencia, almacenamiento y parámetros; ofrece la opción más económica. La gráfica F1/costo permite decidir si la ganancia observada de VGG compensa el salto de cómputo. La gráfica de tiempo incorpora el costo inicial de cache train/validación: ignorarlo favorecería artificialmente al transfer learning. No se presenta un cociente arbitrario F1/FLOPs como una medida universal de calidad.

Transferencia y redimensionamiento

Las convoluciones preentrenadas detectan bordes, texturas y formas reutilizables en imágenes de animales y vehículos. Los bloques superiores son más específicos a ImageNet. El resize aproxima la escala esperada pero solo interpola los 1,024 píxeles originales. Pasar a 224 produce 50,176 píxeles por canal, 49 veces más, y aproximadamente ese crecimiento en costo convolucional, sin multiplicar en la misma proporción el costo del clasificador. 32×32 reduce cómputo pero puede perjudicar desempeño.

Bloques, learning rate y sobreajuste

FT1 B5, LR backbone 0.0001: F1 val 67.70%; FT2 B4+5, LR backbone 0.0001: F1 val 67.54%; FT3 B5, LR backbone 0.0005: F1 val 69.47%. Ganó FT3. Las nueve curvas reducen ambas pérdidas entre primera y última epoch; no se observa esa señal temprana de sobreajuste. La diferencia final loss validación - entrenamiento fue -0.027; dropout y augmentation hacen que esas pérdidas no sean estrictamente comparables. FT2/FT3 usan batch 1024 frente a 512 de FT1; FT3 cambia también dropout, así que no se aíslan completamente estos efectos. Dos épocas VGG no aseguran convergencia ni descartan overfitting posterior. No se comprobó catastrophic forgetting: requiere medir retención de la tarea original o comparar representaciones.

Despliegue

Para un servidor con GPU priorizaríamos VGG-16 fine-tuning si la calidad observada es el objetivo; validaríamos primero su latencia y memoria en la GPU real, pues aquí solo se midió CPU. Para móvil elegiríamos la CNN propia por menor número de parámetros y operaciones. Como alternativa preentrenada consideraríamos MobileNetV3 Small, diseñada con bloques ligeros; su calidad y latencia en este dataset necesitarían un experimento adicional.

Referencias y entrega

[1] Krizhevsky, Learning Multiple Layers of Features from Tiny Images (2009). [2] Torchvision VGG-16. [3] PyTorch Transfer Learning tutorial. [4-6] PyTorch autograd, optim y Module. [7] THOP. [8-9] PyTorch CUDA memory/synchronize. [10] Torchvision MobileNetV3 Small. Enlaces completos consultados: notebook y ANALISIS.md.

Repositorio Git (notebook ejecutado, código y resultados):

https://github.com/iancumes/Laboratorio6DeepLearning

Reproducir: pip install -r requirements.txt; python scripts/run_experiments.py. Las imágenes y pesos se descargan; los resultados medidos y predicciones están versionados. Una sola semilla, resolución reducida y dos o tres epochs limitan la generalización de las conclusiones.